EOSC 213: Computational Methods in Geological Engineering

Lecture 15: Taylor series + Newton's iteration formula

Shadab Ahamed

Department of Earth, Ocean and Atmospheric Sciences,
The University of British Columbia

Thu, Mar 10, 2026

Logistics

Homework 03

- Has been posted.
- Submission deadline: **20 March, 2026** at 11:59 pm
- Solutions will be released about 24 hours after the deadline; **no submission allowed after the solutions have been released**.

Logistics

Homework 03

- Has been posted.
- Submission deadline: **20 March, 2026** at 11:59 pm
- Solutions will be released about 24 hours after the deadline; **no submission allowed after the solutions have been released**.

Midterm II

- Date: **24 March, 2026**
- Will cover material from Lectures 13-18
- Practice Problems Set for Midterm-II (with solutions) will be posted

Plan ahead

Start HW03 early and review implicit methods and BVPs in preparation for Midterm II.

Logistics

Weekly worksheets

- Weekly worksheets containing easy problems related to the materials covered in the week; will be **posted on Tuesdays**.
- **No solutions** will be posted for these (until few days before the final exam).
- **These will not be graded**; this is for your own practice.
- Please solve the worksheet, discuss with your classmates, and attend office hours so you have questions.
- These will also be good practice to get familiar with the material, to **develop problem solving skills** and studying for Midterms and Final exam.

Recap from Lecture 14

Last lecture we discussed **stiff differential equations** and why they motivate implicit methods.
Key ideas from Lecture 14:

- Explicit methods compute the next state directly:

$$y_{n+1} = y_n + hf(t_n, y_n)$$

Recap from Lecture 14

Last lecture we discussed **stiff differential equations** and why they motivate implicit methods.
Key ideas from Lecture 14:

- Explicit methods compute the next state directly:

$$y_{n+1} = y_n + hf(t_n, y_n)$$

- For stiff problems, explicit methods require **very small step sizes** for stability.

Recap from Lecture 14

Last lecture we discussed **stiff differential equations** and why they motivate implicit methods.
Key ideas from Lecture 14:

- Explicit methods compute the next state directly:

$$y_{n+1} = y_n + hf(t_n, y_n)$$

- For stiff problems, explicit methods require **very small step sizes** for stability.
- Implicit methods improve stability but require solving equations.
  Example: **Backward Euler**

$$y_{n+1} = y_n + hf(t_{n+1}, y_{n+1})$$

Recap from Lecture 14

Last lecture we discussed **stiff differential equations** and why they motivate implicit methods.
Key ideas from Lecture 14:

- Explicit methods compute the next state directly:

$$y_{n+1} = y_n + hf(t_n, y_n)$$

- For stiff problems, explicit methods require **very small step sizes** for stability.
- Implicit methods improve stability but require solving equations.
  Example: **Backward Euler**

$$y_{n+1} = y_n + hf(t_{n+1}, y_{n+1})$$

- The unknown y_{n+1} appears on both sides.

Implicit step becomes a root-finding problem

The Backward Euler step:

$$y_{n+1} = y_n + hf(t_{n+1}, y_{n+1})$$

Implicit step becomes a root-finding problem

The Backward Euler step:

$$y_{n+1} = y_n + hf(t_{n+1}, y_{n+1})$$

Define the residual $F(y_{n+1})$:

$$F(y_{n+1}) = y_{n+1} - y_n - hf(t_{n+1}, y_{n+1})$$

Implicit step becomes a root-finding problem

The Backward Euler step:

$$y_{n+1} = y_n + hf(t_{n+1}, y_{n+1})$$

Define the residual $F(y_{n+1})$:

$$F(y_{n+1}) = y_{n+1} - y_n - hf(t_{n+1}, y_{n+1})$$

We must solve

$$F(y_{n+1}) = 0$$

- If f is linear $\rightarrow$ linear system.
- If f is nonlinear $\rightarrow$ nonlinear equation.

Implicit step becomes a root-finding problem

The Backward Euler step:

$$y_{n+1} = y_n + hf(t_{n+1}, y_{n+1})$$

Define the residual $F(y_{n+1})$:

$$F(y_{n+1}) = y_{n+1} - y_n - hf(t_{n+1}, y_{n+1})$$

We must solve

$$F(y_{n+1}) = 0$$

- If f is linear $\rightarrow$ linear system.
- If f is nonlinear $\rightarrow$ nonlinear equation.

So each implicit step becomes a **root-finding problem**.

Root-finding problem

Many numerical problems reduce to solving

$$f(x) = 0$$

Root-finding problem

Many numerical problems reduce to solving

$$f(x) = 0$$

Examples:

- Solving nonlinear algebraic equations
- Implicit time stepping in ODEs
- Nonlinear systems in physics and engineering
- Optimization (finding stationary points)

Root-finding problem

Many numerical problems reduce to solving

$$f(x) = 0$$

Examples:

- Solving nonlinear algebraic equations
- Implicit time stepping in ODEs
- Nonlinear systems in physics and engineering
- Optimization (finding stationary points)

For nonlinear equations, closed-form solutions rarely exist.
Instead, we use **iterative numerical methods**.
One of the most important methods:

Newton's Method

Refresher: Taylor Series Expansion

Taylor series allows us to approximate a function f near a point x_0 using a polynomial function.

Refresher: Taylor Series Expansion

Taylor series allows us to approximate a function f near a point x_0 using a polynomial function. Suppose we know a function and its derivatives at a point x_0.

Refresher: Taylor Series Expansion

Taylor series allows us to approximate a function f near a point x_0 using a polynomial function. Suppose we know a function and its derivatives at a point x_0.

Then near x_0 we can approximate the function using a polynomial expansion.

$$f(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2!}(x - x_0)^2 + \frac{f^{(3)}(x_0)}{3!}(x - x_0)^3 + \dots$$

This expansion becomes more accurate as x gets closer to x_0.

First-order Taylor approximation

If we keep only the first two terms, then it is a **first-order Taylor approximation**.

First-order Taylor approximation

If we keep only the first two terms, then it is a **first-order Taylor approximation**.

$$f(x) \approx f(x_0) + f'(x_0)(x - x_0)$$

First-order Taylor approximation

If we keep only the first two terms, then it is a **first-order Taylor approximation**.

$$f(x) \approx f(x_0) + f'(x_0)(x - x_0)$$

Interpretation:

- We approximate the function using its **tangent line**.
- The approximation is accurate when x is close to x_0.

So locally, a nonlinear function behaves approximately like a **linear function**.

Geometric interpretation

The approximation

$$f(x) \approx f(x_0) + f'(x_0)(x - x_0)$$

is simply the equation of the **tangent line** to the curve at x_0.
Properties:

- The tangent line has slope $f'(x_0)$.
- It touches the curve at x_0.
- Near x_0, the tangent line closely approximates the function.

This idea is extremely important in numerical methods.

Example I - Taylor expansion (first-order approximation)

Consider

$$f(x) = \sin(x)$$

Expand near $x_0 = 0$,

$$f(0) = \sin(0) = 0$$

Example I - Taylor expansion (first-order approximation)

Consider

$$f(x) = \sin(x)$$

Expand near $x_0 = 0$,

$$f(0) = \sin(0) = 0$$

$$f'(x) = \cos(x)$$

$$f'(0) = \cos(0) = 1$$

Example I - Taylor expansion (first-order approximation)

Consider

$$f(x) = \sin(x)$$

Expand near $x_0 = 0$,

$$f(0) = \sin(0) = 0$$

$$f'(x) = \cos(x)$$

$$f'(0) = \cos(0) = 1$$

First-order Taylor approximation:

$$\begin{aligned}\sin(x) &\approx f(x_0) + f'(x_0)(x - x_0) \\ &\approx \sin(0) + \cos(0)(x - 0) = x\end{aligned}$$

This approximation works well when x is small.

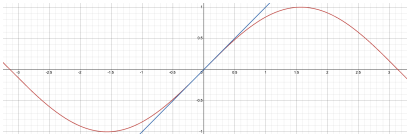


Figure: First-order approximation to sin(x); Red: sin(x), Blue: x

Example I - Taylor expansion (higher-order approximation)

Consider

$$f(x) = \sin(x)$$

Example I - Taylor expansion (higher-order approximation)

Consider

$$f(x) = \sin(x)$$

$$f'(x) = \cos(x); \qquad f''(x) = -\sin(x); \qquad f'''(x) = -\cos(x)$$

$$f'(0) = \cos(0) = 1; \qquad f''(0) = -\sin(0) = 0; \qquad f'''(0) = -\cos(0) = -1$$

Example I - Taylor expansion (higher-order approximation)

Consider

$$f(x) = \sin(x)$$

$$f'(x) = \cos(x); \qquad f''(x) = -\sin(x); \qquad f'''(x) = -\cos(x)$$

$$f'(0) = \cos(0) = 1; \qquad f''(0) = -\sin(0) = 0; \qquad f'''(0) = -\cos(0) = -1$$

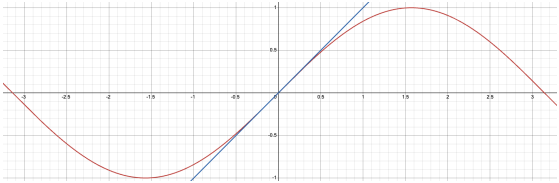
First-order Taylor approximation:

$$\begin{aligned}\sin(x) &\approx f(x_0) + f'(x_0)(x - x_0) + f''(x_0)\frac{(x - x_0)^2}{2!} + f'''(x_0)\frac{(x - x_0)^3}{3!} \\ &\approx \sin(0) + \cos(0)(x - 0) - \sin(0)\frac{(x - 0)^2}{2!} - \cos(0)\frac{(x - 0)^3}{3!} = x - \frac{x^3}{6}\end{aligned}$$

Approximating $\sin(x)$ using Taylor series

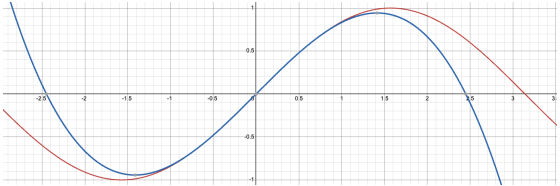
First-order approximation

$$\sin(x) \approx x$$



Third-order approximation

$$\sin(x) \approx x - \frac{x^3}{6}$$



Figures plotted using Desmos. Feel free to check it out.

Example II - Taylor expansion (first-order approximation)

Consider

$$f(x) = \cos(x)$$

Expand near $x_0 = 0$,

$$f(0) = \cos(0) = 1$$

Example II - Taylor expansion (first-order approximation)

Consider

$$f(x) = \cos(x)$$

Expand near $x_0 = 0$,

$$f(0) = \cos(0) = 1$$

$$f'(x) = -\sin(x)$$

$$f'(0) = -\sin(0) = 0$$

Example II - Taylor expansion (first-order approximation)

Consider

$$f(x) = \cos(x)$$

Expand near $x_0 = 0$,

$$f(0) = \cos(0) = 1$$

$$f'(x) = -\sin(x)$$

$$f'(0) = -\sin(0) = 0$$

First-order Taylor approximation:

$$\begin{aligned}\cos(x) &\approx f(x_0) + f'(x_0)(x - x_0) \\ &\approx \cos(0) - \sin(0)(x - 0) = 1\end{aligned}$$

This approximation works well when x is small.

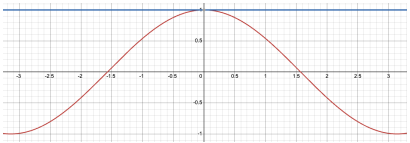


Figure: First-order approximation to cos(x); Red: cos(x), Blue: 1

Example II - Taylor expansion (higher-order approximation)

Consider

$$f(x) = \cos(x)$$

Example II - Taylor expansion (higher-order approximation)

Consider

$$f(x) = \cos(x)$$

$$f'(x) = -\sin(x); \qquad f''(x) = -\cos(x); \qquad f'''(x) = \sin(x)$$

$$f'(0) = -\sin(0) = 0; \qquad f''(0) = -\cos(0) = -1; \qquad f'''(0) = \sin(0) = 0$$

Example II - Taylor expansion (higher-order approximation)

Consider

$$f(x) = \cos(x)$$

$$f'(x) = -\sin(x); \qquad f''(x) = -\cos(x); \qquad f'''(x) = \sin(x)$$

$$f'(0) = -\sin(0) = 0; \qquad f''(0) = -\cos(0) = -1; \qquad f'''(0) = \sin(0) = 0$$

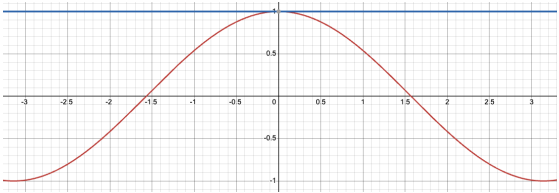
First-order Taylor approximation:

$$\begin{aligned}\cos(x) &\approx f(x_0) + f'(x_0)(x - x_0) + f''(x_0)\frac{(x - x_0)^2}{2!} + f'''(x_0)\frac{(x - x_0)^3}{3!} \\ &\approx \cos(0) - \sin(0)(x - 0) - \cos(0)\frac{(x - 0)^2}{2!} + \sin(0)\frac{(x - 0)^3}{3!} = 1 - \frac{x^2}{2}\end{aligned}$$

Approximating $\cos(x)$ using Taylor series

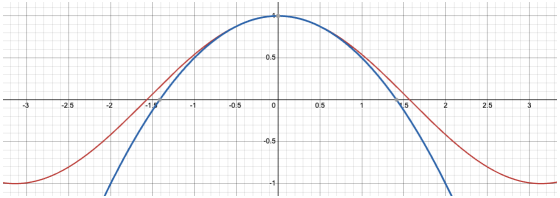
First-order approximation

$$\cos(x) \approx 1$$



Second-order approximation

$$\cos(x) \approx 1 - \frac{x^2}{2}$$



Figures plotted using Desmos. Feel free to check it out.

Maclaurin Series Expansions

Taylor series expansion around $x = 0$ is called the **Maclaurin series**.

Sine function

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \frac{x^9}{9!} - \cdots = \sum_{n=0}^{\infty} (-1)^n \frac{x^{2n+1}}{(2n+1)!}$$

Cosine function

$$\cos(x) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \frac{x^8}{8!} - \cdots = \sum_{n=0}^{\infty} (-1)^n \frac{x^{2n}}{(2n)!}$$

Tangent function

$$\tan(x) = x + \frac{x^3}{3} + \frac{2x^5}{15} + \frac{17x^7}{315} + \frac{62x^9}{2835} + \cdots$$

Why Taylor approximation is useful

Taylor expansions are used throughout numerical methods.
Examples:

- Deriving numerical differentiation formulas
- Deriving numerical integration methods
- Constructing ODE solvers (Euler methods)
- Approximating nonlinear equations locally

In many algorithms we replace a complicated nonlinear function by a **local linear approximation**.

Key idea we will use today

Suppose we want to solve

$$f(x) = 0$$

If the function is nonlinear, solving this equation directly may be difficult.
Instead we:

- Start with a guess x_k
- Use a **first-order Taylor approximation**
- Replace the nonlinear function by a **linear approximation**

This idea leads directly to **Newton's Method**.

Idea behind Newton's Method

Suppose we want to solve

$$f(x) = 0$$

Idea behind Newton's Method

Suppose we want to solve

$$f(x) = 0$$

Start with an initial guess

$$x_0$$

Idea behind Newton's Method

Suppose we want to solve

$$f(x) = 0$$

Start with an initial guess

$$x_0$$

We approximate the function locally using a **linear Taylor expansion**.

$$f(x) \approx f(x_k) + f'(x_k)(x - x_k)$$

We then find where this linear approximation crosses zero.

$$f(x_k) + f'(x_k)(x - x_k) = 0$$

Derivation of Newton's iteration

Using the linear approximation

$$f(x) \approx f(x_k) + f'(x_k)(x - x_k)$$

Set the approximation to zero:

$$0 = f(x_k) + f'(x_k)(x_{k+1} - x_k)$$

Solve for x_{k+1}:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

This gives the Newton iteration formula:

$$\boxed{x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}}$$

Geometric interpretation

Newton's method uses the tangent line.
Steps:

1. Start with guess x_k
2. Compute tangent line to $f(x)$ at x_k
3. Find where that tangent line crosses the x-axis
4. That becomes the next guess x_{k+1}

Each iteration ideally moves us closer to the true root.

Example 1: Solving by hand

Solve

$$x^2 - 2 = 0$$

The root is $\sqrt{2}$.
Define

$$f(x) = x^2 - 2$$

Derivative:

$$f'(x) = 2x$$

Newton update:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} = x_k - \frac{x_k^2 - 2}{2x_k}$$

Example 1: Newton iterations

Start with initial guess

$$x_0 = 1$$

Iteration 1:

$$x_1 = 1 - \frac{1^2 - 2}{2(1)}$$

$$x_1 = 1.5$$

Iteration 2:

$$x_2 = 1.5 - \frac{1.5^2 - 2}{2(1.5)}$$

$$x_2 = 1.4167$$

True value: $\sqrt{2} \approx 1.4142$. Newton's iteration converges very quickly.

Example 2: Quadratic equation with two real roots

Consider the equation

$$f(x) = x^2 - 5x + 6 = 0$$

This factors as

$$(x-2)(x-3) = 0$$

So the roots are

$$x = 2, \quad x = 3$$

Derivative:

$$f'(x) = 2x - 5$$

Newton iteration:

$$x_{k+1} = x_k - \frac{x_k^2 - 5x_k + 6}{2x_k - 5}$$

We will now apply Newton's method with two different initial guesses:

- $x_0 = 1.8$ (close to root $x = 2$)
- $x_0 = 3.2$ (close to root $x = 3$)

Dependence on the initial guess

Newton's method converges to the root closest to the initial guess.

- Starting with $x_0 = 1.8$
  Iterations move toward

$$x = 2$$

- Starting with $x_0 = 3.2$
  Iterations move toward

$$x = 3$$

Key idea

Newton's method typically converges to the root that lies **closest to the initial guess**.

Example 3: Higher-order polynomials

Consider the equation

$$f(x) = x^3 - x - 2 = 0$$

This equation does not have a simple closed-form root.
Derivative:

$$f'(x) = 3x^2 - 1$$

Newton iteration becomes

$$x_{k+1} = x_k - \frac{x_k^3 - x_k - 2}{3x_k^2 - 1}$$

We will typically solve such problems using **code**.

Newton method algorithm (pseudocode)

Given a function $f(x)$:

1. Choose initial guess x_0
2. For $k = 0, 1, 2, \ldots$

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

3. Stop when

$$|x_{k+1} - x_k| < \epsilon$$

In practice, this algorithm is implemented computationally.

Connection to implicit methods

Recall the Backward Euler step:

$$y_{n+1} = y_n + hf(t_{n+1}, y_{n+1})$$

Define

$$g(y_{n+1}) = y_{n+1} - y_n - hf(t_{n+1}, y_{n+1})$$

We must solve

$$g(y_{n+1}) = 0$$

Newton's method provides an efficient way to compute y_{n+1}.
This is how nonlinear implicit ODE solvers are implemented.

Limitations of Newton's Method

Newton's method is powerful but has limitations.

- It requires computing the derivative $f'(x)$.
- Convergence depends on a good initial guess.
- It typically converges to **one root near the initial guess**.
- If the initial guess is poor, the method may diverge.
- If $f'(x_k)$ is close to zero, the update can become unstable.

Despite these issues, Newton's method remains one of the most widely used numerical algorithms.

Summary

- Implicit methods require solving equations at each time step.
- These equations often take the form

$$g(x) = 0$$

- Newton's method is a powerful iterative technique for solving such nonlinear equations.
- The update rule is

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

- In the next lecture we will connect this to
  - implicit ODE solvers
  - numerical implementation
  - solving nonlinear systems